Strategic Research Foundation Document: Viability, Methodology, and Execution Plan for MCP-Enabled Agentic Security

Executive Decision

A comprehensive, evidence-backed strategic evaluation of the proposed research direction yields a definitive decision: **REVISE**.

The underlying subject matter—security vulnerabilities within Model Context Protocol (MCP)-enabled agentic systems—represents one of the most urgent, high-impact, and underexplored frontiers in contemporary cybersecurity architecture.[1] The rapid transition of artificial intelligence from reactive text generation to autonomous, goal-seeking execution via standardized tool-calling protocols has fundamentally altered the threat landscape.[1] This paradigm shift has created structural security gaps that current defense mechanisms, which rely heavily on prompt engineering and semantic alignment, are largely failing to mitigate.[2] As autoregressive models process code and data uniformly through the same attention mechanism, they lack the intrinsic ability to distinguish between legitimate system instructions and adversarial payloads injected through external data sources.[2]

The original research proposal correctly identifies two highly specific and advanced attack vectors that exploit this new architecture: privilege aggregation and transitive trust exploits.[1] However, a strict and immediate authorization to proceed cannot be recommended without critical revisions to the execution strategy. There exists a fundamental mismatch between the original research plan's broad conceptual ambitions and the rigid operational constraints inherent to an undergraduate research environment operating with zero funding, restricted consumer-grade hardware, and a mandate for no-cost, non-physical publication.[1]

To ensure this project culminates in a peer-reviewed publication rather than languishing as an unfinished, overly broad experiment, the framework must be rigorously restructured. The entire research strategy must abandon the synthesis of custom evaluation datasets, eliminate the investigation of "Transitive Trust," and pivot exclusively toward a highly focused empirical evaluation of multi-tool "Privilege Aggregation" using established open-source benchmarks.[1] Furthermore, the experimental design must enforce a strict baseline of cognitive competence to account for the inherent reasoning limitations of local, sub-10B parameter models.[1] By implementing these revisions, the core scientific inquiry becomes exceptionally strong, technically feasible, and highly optimized for publication in hybrid-access, non-APC academic journals.

Extracted Current Idea

The foundational documents and supplementary materials outline a highly ambitious research

initiative targeting the intersection of language model autonomy and standardized protocol security.[1]

The Current Research Niche

The research targets the security of Agentic AI systems connected via the Model Context Protocol (MCP).[1] The focus is specifically on indirect prompt injections and the subsequent cognitive hijacking of large language models when exposed to a suite of external tools and untrusted data sources.[1] MCP, recently introduced as a standardized JSON-RPC integration layer, separates the AI application from the actual tool execution environment using a formal client-host-server paradigm.[1] The niche explores how this standardization inadvertently creates standardized attack vectors.

Proposed Titles

The primary working title extracted from the initial proposal is: "Privilege Aggregation and Transitive Trust Exploits in Model Context Protocol (MCP)-Enabled Agentic AI Systems".[1]

Core Research Problem

The fundamental problem addresses how the architectural standardization of AI tool usage via MCP introduces severe exploit chains.[1] Specifically, the research questions whether autoregressive language models can be manipulated into executing unauthorized actions when exposed to complex external environments.[1] The initial proposal identifies two distinct mechanisms:

1. **Privilege Aggregation:** This occurs when multiple individually secure tools (e.g., a benign file reader and a benign network webhook) are exposed to a single agent, creating a combined capability space that violates the principle of least privilege.[1]
2. **Transitive Trust:** This describes the implicit inheritance of authorization across a chain of systems, where an AI agent assumes the trustworthiness of a remote resource (such as a Git repository or a Sentry error log), allowing malicious data to propagate back up the chain and command the high-privilege agent.[1]

Claimed Novelty

The proposed novelty resides in shifting the academic focus away from static, single-tool poisoning or traditional software bugs (such as raw SQL injection within an MCP server) toward the cognitive mechanics of multi-hop tool-chaining.[1] The research argues that evaluating the propensity of an LLM to aggregate privileges maliciously under adversarial pressure within an isolated MCP environment remains genuinely underexplored in peer-reviewed journals.[1]

Proposed Methodology

The initial methodological approach suggests creating a sandboxed local environment utilizing Docker, local open-weight LLMs, and custom-built vulnerable MCP Python servers utilizing the FastMCP framework.[1] The plan involves generating custom injection payloads to measure

Attack Success Rate (ASR) and Tool Invocation Deviation (TID) across various levels of tool exposure.[1]

Datasets, Tools, and Models

The proposal relies heavily on free and open-source infrastructure.[1] It proposes the utilization of local open-weight models in the 8-billion parameter class, executed entirely offline via the Ollama framework.[1] The architectural implementation depends on Python 3, Docker for containerized isolation, and the official Anthropic MCP Python SDK.[1] Initially, the proposal suggests synthesizing custom evaluation datasets containing specialized injection vectors.[1]

Publication Strategy

The initial strategy heavily emphasizes top-tier academic conferences (such as ACM CCS, USENIX Security, or IEEE S&P) and specialized physical workshops.[1]

Comprehensive Constraints Audit

An exhaustive audit of the current direction against the stated undergraduate constraints reveals critical deviations that mandate immediate structural revision:

1. **Scope Overreach and Mechanism Conflation:** The current direction is excessively broad. Investigating both "Privilege Aggregation" and "Transitive Trust" simultaneously dilutes the experimental focus. Evaluating Transitive Trust requires simulating remote network topologies and spoofing external data sources (like Git or Sentry) [1], which introduces immense networking complexity for a local setup. Conversely, Privilege Aggregation is mathematically elegant and highly controllable; it requires only scaling the number of available local tools in an experiment to measure cognitive degradation.[1] The final paper must abandon Transitive Trust and focus strictly on Privilege Aggregation.
2. **Publication Misalignment:** The proposed publication strategy is fundamentally flawed under the given constraints. Top-tier conferences universally require in-person attendance, physical presentations, and substantial travel budgets, which explicitly violates the no-cost, no-travel constraint.[1] The publication strategy must be completely reoriented toward reputable, indexed journals offering zero-APC (Article Processing Charge) hybrid subscription routes.[1]
3. **Methodological Risk Regarding Datasets:** Synthesizing a custom dataset for multi-tool chaining is too time-consuming, introduces massive author bias, and risks immediate rejection by peer reviewers.[1] The execution must absolutely rely on emerging, standardized open-source benchmarks, such as AgentDojo or the Open Agent Security Benchmark (OASB), which already possess peer-reviewed security test cases.[1]
4. **Hardware and Cognitive Bottlenecks:** Operating sub-10B parameter models locally poses a severe risk of "tool-calling hallucination".[1] Small models frequently fail to output valid JSON-RPC schemas or hallucinate tool parameters even during benign tasks.[1] If the model fails during an adversarial attack simply because it lacks the cognitive capacity to format JSON, the resulting Attack Success Rate data will contain massive false negatives,

invalidating the study. A mandatory competence baseline evaluation must be enforced before any security testing begins.[1]

Literature Review Map

A targeted investigation into peer-reviewed literature, high-impact arXiv preprints, and industry technical reports from 2023 to 2026 highlights the rapid evolution of agentic security. The field is currently experiencing a massive pivot from standard direct prompt injection (DPI) toward semantic tool manipulation and protocol exploitation. The following analysis maps the 14 most critical papers that contextualize the proposed research, delineate the exact academic gap, and inform the experimental boundaries.

Full Title	Authors & Year	Venue / Link	Core Contribution	Relevance & Remaining Gap
Trustworthy Agentic AI Requires Deterministic Architectural Boundaries	Bhattarai & Vu (2026)	arXiv [2]	Formalizes the "Lethal Trifecta" (untrusted inputs, privileged data, external action) and proposes the Trinity Defense architecture.	Provides the core theoretical justification for the study, arguing alignment is insufficient. However, it is largely theoretical and lacks consumer-grade protocol empirical data.[1]
Breaking the Protocol: Security Analysis of the Model Context Protocol Specification...	Maloyan & Namiot (2026)	arXiv [6]	Performs a formal security analysis of MCP v1.0. Identifies flaws like absence of capability attestation and bidirectional sampling without origin	Highly relevant. Establishes that MCP vulnerabilities are architectural. Leaves a gap in evaluating the cognitive limits of small, edge-deploye

			authentication.[6]	d LLMs when confronted with these flaws.[6]
Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection...	Huang et al. (2026)	arXiv [12]	Provides a STRIDE/DREAD threat model of MCP. Focuses heavily on "tool poisoning" where malicious instructions are embedded in tool metadata schemas.[13]	Threatens novelty slightly regarding tool poisoning, but leaves a massive gap regarding dynamic, multi-hop privilege aggregation and multi-tool scaling.[1]
AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents	Debenedetti et al. (2024)	arXiv [4]	Introduces an evaluation framework featuring 97 realistic tasks and 629 security test cases specifically for agents executing tools over untrusted data.[4]	Strengthens experimental feasibility. Replacing custom datasets with AgentDojo solves the timeline constraint while providing rigorous peer-reviewed test cases.[1]
Agent Security Bench (ASB): A Comprehensive Benchmark to Evaluate	Zhang et al. (2025)	ICLR 2025 / arXiv [9]	Introduces ASB to test agent robustness in multi-stage tool-use settings, critiquing prior	Provides an alternative or complementary benchmark to AgentDojo. Validates that multi-tool

the Security of LLM Agents			benchmarks for overriding agent planning decisions.[15]	chaining evaluations are a primary frontier in agentic security.[9]
VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers	Sun et al. (2026)	arXiv [16]	Presents an automated framework that discovered 106 0-day vulnerabilities in MCP servers using static analysis and prompt evolution.[16]	Focuses strictly on server-side taint vulnerabilities (e.g., raw SQL/bash injection in server code) rather than LLM cognitive semantic manipulation.[19]
Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis...	Maloyan et al. (2026)	arXiv [20]	Synthesizes 78 studies, revealing 85% of attacks compromise major platforms. Notes that protocol-level and tool poisoning attacks are underappreciated.[20]	Justifies the high impact of the research. Confirms that protocol-level manipulation (the exact focus of this study) is currently an under-secured ecosystem.[20]
CSA Research Note: Agentjacking - Sentry MCP Injection Hijacks AI	Tenet Security / CSA (2026)	CSA Labs [7]	Documents a real-world exploit where attackers inject fake error events into	Provides the ultimate real-world case study for "Transitive Trust."

Coding Agents			Sentry via public DSNs, which the AI agent then blindly executes via MCP.[7]	Demonstrates that tools behaving benignly can be turned malicious in production environments.[2] [1]
ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark...	Pang et al. (2024)	NAACL 2025 [22]	Proposes a stateful benchmark for LLM tool capabilities, emphasizing that agent logic is manipulable through data, unlike deterministic microservices.[2] [2]	Reinforces the difficulty of measuring true agent capability. Highlights the need to establish benign operational baselines before adversarial testing.[23]
LiveMCPBench: Can Agents Navigate an Ocean of MCP Tools?	Mo et al. (2025)	arXiv [24]	Constructs benchmarks evaluating agents' ability to orchestrate multiple tools, specifically utilizing the Model Context Protocol.[24]	Indicates that MCP benchmarking is currently a highly competitive sub-field. Increases urgency for publication but provides excellent comparative methodologies.[24]
AgentDyn: A	Lee et al.	arXiv [25]	Criticizes static	Highlights the

Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks...	(2026)		benchmarks for lacking dynamic open-ended tasks and introduces 60 tasks and 560 injection cases.[25]	limitation of static validation. Suggests that the experimental design must involve multi-turn agent loops to be academically credible.[25]
InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents	Zhan et al. (2024)	arXiv [26]	Formalizes IPI attacks on tool-integrated LLM agents and introduces a baseline benchmark covering domains like financial harm and data security.[26]	Represents the foundational generation of benchmarks, though newer studies (like ASB) critique its methodology for overriding planning constraints.[15]
ComplexMCP: A Benchmark for Evaluating Agents in Rigorous Conditions	Yan et al. / Wang et al. (2025)	arXiv [24]	Evaluates agents orchestrating multiple tools in complex conditions via MCP, noting the limitations of previous benchmarks.[24]	Confirms that multi-tool orchestration via MCP is currently the primary evaluation frontier, strengthening the project's academic relevance.[24]
Enterprise-Grade Security	Anonymous (2025)	Industry Whitepaper [28]	Translates theoretical	Highlights enterprise

for the Model Context Protocol (MCP)			security concerns into an actionable framework, focusing on zero-trust architectures and API security.[28]	demand for MCP security research, ensuring high citation potential for rigorous empirical evaluations.[28]

Narrative Synthesis of Literature and Gap Identification

The literature review reveals a clear chronological evolution in agentic security research. Initially, studies focused on direct prompt injections designed to bypass content filters. As models evolved into active agents, research like that of Zhan et al. with the InjecAgent benchmark began formalizing Indirect Prompt Injections (IPI) within tool-integrated systems.[26] However, recent critiques by Zhang et al. regarding the Agent Security Bench (ASB) indicate that early benchmarks artificially forced tool injections, overriding the agent's internal planning mechanisms and skewing empirical data.[9] Consequently, modern evaluations require dynamic, stateful environments, as pioneered by Debenedetti et al. with AgentDojo [4] and Pang et al. with ToolSandbox.[22]

Simultaneously, the theoretical understanding of why agents fail has crystallized. Bhattarai and Vu established the "Lethal Trifecta"—the inherent vulnerability created when untrusted inputs, privileged data access, and external action capabilities are combined within an autoregressive architecture that cannot deterministically separate command tokens from data tokens.[2] Maloyan and Namiot applied this specifically to the Model Context Protocol, proving that MCP's architectural reliance on unverified capability declarations and bidirectional sampling without origin authentication amplifies these vulnerabilities by 23–41% compared to traditional APIs.[6] This theoretical risk was emphatically validated in the wild by Tenet Security's discovery of "Agentjacking," where attackers weaponized public Sentry Data Source Names (DSNs) to feed malicious markdown into AI coding assistants via MCP, achieving remote code execution with the developer's local privileges.[7]

However, a careful analysis of the most recent computational security auditing literature reveals a massive, exploitable gap for an undergraduate researcher. The state-of-the-art VIPER-MCP framework, which scanned nearly 40,000 GitHub repositories and discovered 106 zero-day vulnerabilities in MCP servers, relies entirely on "taint-style" static analysis.[16] VIPER-MCP searches for traditional software bugs—such as poorly sanitized inputs being passed directly to subprocess.run() in Python.[16]

The Missing Gap: VIPER-MCP and similar auditing tools completely ignore the *semantic cognitive vulnerability* of the LLM itself. The literature confirms that individual MCP tools can be poisoned and that traditional code bugs exist in servers. Yet, there remains a glaring scarcity of quantitative, mathematically modeled research detailing dynamic multi-hop privilege

aggregation. Specifically, no peer-reviewed empirical study currently charts the degradation of cognitive security (the Attack Success Rate) in edge-deployed (sub-10B parameter) models as the *volume of concurrent benign tools scales*. The field urgently needs to understand the mathematical threshold at which exposing multiple benign capabilities transforms a local agent into a critical system vulnerability. This specific intersection—evaluating the cognitive propensity of an LLM to chain tools maliciously under scaling adversarial pressure within an isolated MCP environment—is the exact gap this research will exploit.

Candidate Research Directions

Based on the synthesis of the foundational documents and the extracted literature, the following candidate research directions represent distinct variations within the identified MCP security niche.

Candidate 1: Privilege Aggregation Scaling in Edge-Deployed Agents

- **Candidate Title:** Empirical Evaluation of Privilege Aggregation Vulnerabilities in Edge-Deployed Open-Weight Agents via the Model Context Protocol.
- **Core Research Question:** At what threshold of tool exposure does a sub-10B parameter local LLM lose the cognitive capacity to resist indirect prompt injection, resulting in unauthorized privilege aggregation?
- **Main Contribution:** A quantitative mathematical model charting Attack Success Rate (ASR) and Tool Invocation Deviation (TID) against tool density scaling (e.g., 1 tool vs. 3 tools vs. 5 tools).
- **Required Datasets / Benchmarks:** AgentDojo multi-tool injection test cases.[4]
- **Required Tools / Models:** Docker, FastMCP Python library, Ollama, Hermes-2-Pro-Llama-3-8B.
- **Compute Requirement:** Consumer-grade laptop (16GB RAM, local execution).[1]
- **Expected Experimental Output:** Line graphs demonstrating a statistically significant increase in ASR as the number of available tools increases, proving capability scaling risk.
- **Publication Fit:** Excellent fit for practice-driven, empirical journals like the *Journal of Information Security and Applications*.
- **Main Risks:** The 8B model may fail to format JSON properly, ruining the baseline evaluation.[1]
- **Estimated Feasibility for Undergraduate:** Exceptionally high, as it requires zero complex networking and relies entirely on localized Python array simulations.
- **Novelty Strength:** High; shifts focus from static tool poisoning to dynamic capability scaling.

Candidate 2: Transitive Trust Hijacking via Version Control MCPs

- **Candidate Title:** Measuring Transitive Trust Propagation in Agentic Workflows: A Case Study on Git-Enabled MCP Servers.
- **Core Research Question:** How effectively can malicious markdown embedded in remote Git repositories hijack local agentic execution loops via standardized MCP implementations?

- **Main Contribution:** Empirical validation of the "Agentjacking" concept using open-weight models, mapped against the Lethal Trifecta framework.
- **Required Datasets / Benchmarks:** Custom Git repositories seeded with specific indirect prompt injection payloads.
- **Required Tools / Models:** Docker, Git MCP Server, open-weight LLMs via Ollama.
- **Compute Requirement:** Local laptop with stable internet for repository cloning.
- **Expected Experimental Output:** Empirical success rates of arbitrary local code execution achieved solely through the agent reading remote, unprivileged repositories.
- **Publication Fit:** Good fit for applied security and networking journals.
- **Main Risks:** Networking complexity; managing stateful Git environments inside Docker containers can lead to difficult debugging loops for an undergraduate.
- **Estimated Feasibility for Undergraduate:** Moderate.
- **Novelty Strength:** Moderate-High; heavily builds upon the recently publicized Tenet Security Agentjacking research.[21]

Candidate 3: Evaluating Automated Static Defense Mechanisms for MCP Tool Schemas

- **Candidate Title:** Mitigating Tool Poisoning in MCP Environments through Deterministic JSON Schema Validation.
- **Core Research Question:** Can client-side programmatic reference monitors reduce ASR below 5% without severely degrading the agent's benign task utility?
- **Main Contribution:** The development and evaluation of an open-source Python interceptor wrapper for MCP clients that neutralizes schema-level prompt injections via static analysis.
- **Required Datasets / Benchmarks:** Open Agent Security Benchmark (OASB).[9]
- **Required Tools / Models:** Python AST manipulation libraries, FastMCP, open-weight models.
- **Compute Requirement:** Local laptop.
- **Expected Experimental Output:** Comparative analysis tables of ASR and Utility Score (US) with and without the defensive wrapper engaged.
- **Publication Fit:** Strong fit for software engineering and formal methods journals.
- **Main Risks:** High software engineering burden; building a defense that actually works without breaking the JSON-RPC pipeline is exceedingly difficult.
- **Estimated Feasibility for Undergraduate:** Moderate-Low.
- **Novelty Strength:** High, directly answering the call for deterministic boundaries proposed by the Trinity Defense.[3]

Candidate 4: Generating a Custom Evaluation Dataset for Multi-Agent MCP Scenarios

- **Candidate Title:** AgentExploit: A Novel Dataset for Evaluating Transitive Trust in Multi-Agent Frameworks.
- **Core Research Question:** What constitutes a statistically rigorous, unbiased dataset for

measuring cross-agent prompt injection via shared MCP resources?
- **Main Contribution:** The synthesis, validation, and release of a comprehensive 5,000-sample dataset designed specifically for testing complex MCP interactions.
- **Required Datasets / Benchmarks:** Ground-up generation via programmatic LLM prompting.
- **Required Tools / Models:** Paid API access (e.g., GPT-4o) to generate and self-validate high-quality diverse prompts.
- **Compute Requirement:** Cloud computing for mass generation and validation.
- **Expected Experimental Output:** A published JSON dataset and baseline evaluation metrics.
- **Publication Fit:** Data-centric journals or benchmark tracks at conferences.
- **Main Risks:** Extreme time requirements; massive risk of reviewer rejection due to unproven validity compared to established benchmarks.
- **Estimated Feasibility for Undergraduate:** Extremely Low.
- **Novelty Strength:** Moderate; the field is rapidly saturating with benchmarks (e.g., ASB, LiveMCPBench, ComplexMCP).[9]

Candidate 5: Zero-Day Vulnerability Auditing of Public MCP Registries

- **Candidate Title:** A Large-Scale Security Audit of Community-Contributed Model Context Protocol Servers.
- **Core Research Question:** What percentage of open-source MCP servers on public registries contain exploitable taint-style code vulnerabilities?
- **Main Contribution:** Systematic auditing of thousands of GitHub repositories for traditional software flaws within MCP wrappers.
- **Required Datasets / Benchmarks:** Scraping scripts for public GitHub MCP repositories.
- **Required Tools / Models:** Proprietary or enterprise-grade Static Application Security Testing (SAST) tools, cloud execution environments.
- **Compute Requirement:** High; massive parallel processing required to scan thousands of codebases.
- **Expected Experimental Output:** A catalog of discovered CVEs and zero-day vulnerabilities.
- **Publication Fit:** Vulnerability disclosure and systems security journals.
- **Main Risks:** The research has already been executed.
- **Estimated Feasibility for Undergraduate:** Low.
- **Novelty Strength:** Zero. The VIPER-MCP paper published in May 2026 successfully audited 39,884 repositories and discovered 106 zero-days.[16] Replicating this is entirely redundant.

Candidate 6: Cross-Platform MCP Client Spoofing

- **Candidate Title:** Exploiting the Client-Host Boundary: Spoofing Authorization in Commercial MCP Integrations.
- **Core Research Question:** Can a malicious local process spoof an authorized MCP client

to hijack connections to commercial applications like Claude Desktop or Cursor IDE?
- **Main Contribution:** Penetration testing of closed-source commercial applications utilizing the MCP standard.
- **Required Datasets / Benchmarks:** None. Penetration testing methodology.
- **Required Tools / Models:** Reverse engineering tools, paid subscriptions to commercial AI assistants.
- **Compute Requirement:** Local workstation.
- **Expected Experimental Output:** Proof-of-concept exploits demonstrating authorization bypasses.
- **Publication Fit:** Highly specialized offensive security workshops.
- **Main Risks:** Relies on exploiting closed-source, proprietary software that may patch the vulnerability before the paper is published, rendering the research instantly obsolete.
- **Estimated Feasibility for Undergraduate:** Moderate.
- **Novelty Strength:** High, but highly volatile.

Hard Rejection Filters

The candidate directions must be filtered through the strict operational constraints of the project. Any candidate failing these constraints must be categorically rejected.

- **Candidate 4 is REJECTED (NO-GO):** Fails the custom dataset creation filter and the paid API filter. An undergraduate researcher lacks the time and financial resources (paid GPT-4 API calls) to synthesize and validate a 5,000-sample dataset against established standards like AgentDojo or ASB.[1] Reviewers heavily scrutinize bespoke datasets for author bias, posing a massive risk of total failure.
- **Candidate 5 is REJECTED (NO-GO):** Fails the novelty filter and the compute affordability filter. The VIPER-MCP research [16] has completely saturated this specific angle by scanning nearly 40,000 repositories using advanced dual-mutator scheduling. Competing with this level of institutional compute as a solo undergraduate is impossible.
- **Candidate 6 is REJECTED (NO-GO):** Fails the proprietary tools filter and the risk of total failure filter. Relying on reverse engineering closed-source applications (like Claude Desktop) means Anthropic could deploy a patch mid-experiment, instantly destroying the reproducibility and publishability of the findings.
- **Candidate 3 is REJECTED (REVISE):** Building a novel defense architecture (such as the proposed Trinity Defense reference monitors [3]) requires advanced formal verification and deep architectural integration. It carries a high risk of total failure if the defense inadvertently breaks the JSON-RPC pipeline, degrading the agent's utility so severely that the defense becomes practically useless.

This rigorous filtration leaves **Candidate 1 (Privilege Aggregation Scaling)** and **Candidate 2 (Transitive Trust via Git)** as the sole viable options for comprehensive scoring.

Candidate Scoring Matrix

The remaining candidates are scored out of 10 based on the strict constraints of a zero-budget undergraduate project targeting non-APC journal publication. The scoring is strict to ensure the final selection guarantees execution viability.

Metric	Candidate 1: Privilege Aggregation Scaling	Candidate 2: Transitive Trust via Git MCP
Novelty	9.0	6.5
Literature Gap Clarity	9.0	7.0
Experimental Feasibility	9.5	7.0
Dataset/Benchmark Availability	9.0	6.0
Compute Affordability	9.5	9.5
Implementation Difficulty	7.5	6.0
Evaluation Clarity	9.0	8.0
Reproducibility	9.5	8.0
Publication Potential	8.5	7.0
Undergraduate Suitability	8.5	7.0
Risk of Total Failure (Inverted)	8.0	6.5
Journal Venue Fit	9.0	8.5
CV / Portfolio Value	9.0	8.5
Weighted Final Score	**88.5 / 100 (GO)**	**73.5 / 100 (REVISE)**

Narrative Justification of Scoring: Candidate 1 emerges as the definitive strategic choice. Its exceptionally high score in Novelty (9.0) and Literature Gap Clarity (9.0) stems from the fact that no existing paper explicitly charts the mathematical scaling of ASR against tool volume in sub-10B models. While Candidate 2 is fascinating, the "Agentjacking" concept via remote data

sources has recently been heavily publicized by Tenet Security [21], significantly reducing its academic novelty (6.5).
Furthermore, Candidate 1 dominates in Experimental Feasibility (9.5) and Implementation Difficulty (7.5). Simulating local filesystem tools and databases using Python dictionaries within FastMCP is vastly simpler and more robust for an undergraduate than configuring networked Git repository spoofing across multiple Docker containers, which Candidate 2 requires. Candidate 1 perfectly aligns with the AgentDojo benchmark [4], virtually eliminating the risk of total execution failure by ensuring the methodology rests on a peer-reviewed foundation.

Final Selected Title

Final Recommended Title:
Empirical Evaluation of Privilege Aggregation Vulnerabilities in Edge-Deployed Open-Weight Agents via the Model Context Protocol
Alternative Title Options:

1. Measuring Cognitive Security Boundaries: Multi-Tool Exploit Chaining in MCP-Enabled Open-Weight Models
2. Characterizing the Lethal Trifecta: An Empirical Study of Privilege Aggregation in Sub-10B Parameter AI Agents
3. Tool-Chaining Vulnerabilities in the Model Context Protocol: A Quantitative Analysis of Edge-Deployed LLMs

Strategic Rationale for Title Selection: The recommended title mimics the standard, highly objective academic phrasing preferred by major physical sciences and engineering publishers (such as Elsevier or Wiley).[1] Crucially, it immediately justifies the hardware limitation by framing it as a focused study of "Edge-Deployed Open-Weight Agents." Rather than appearing as a project that simply couldn't afford to run GPT-4, the title defines edge-deployment as a specific, highly relevant research niche driven by enterprise data privacy requirements. Furthermore, it explicitly names the exact attack vector ("Privilege Aggregation") and the standardized framework ("Model Context Protocol"), ensuring maximal search engine optimization for academic indexing and discovery.

Research Problem Formulation

Research Gap: While current security literature has extensively mapped the architectural protocol flaws of the Model Context Protocol (e.g., Maloyan & Namiot, 2026 [10]) and demonstrated static individual tool poisoning (Huang et al., 2026 [12]), there is a critical absence of quantitative empirical data regarding the cognitive collapse of local, open-weight LLMs when forced to navigate complex multi-tool environments. Furthermore, while taint-style code vulnerabilities in MCP servers are well documented (e.g., VIPER-MCP [16]), the literature lacks mathematical modeling of how the Attack Success Rate (ASR) of semantic indirect prompt injections scales in direct proportion to the volume of benign tools exposed to a single agentic context window.
Final Research Problem Statement:
The integration of multiple, individually secure tools through the Model Context Protocol creates a combined capability space that local, sub-10B parameter LLMs are cognitively

incapable of securing against indirect prompt injections. This research addresses the problem of *privilege aggregation*, empirically measuring the mathematical threshold at which exposing multiple benign capabilities transforms an edge-deployed agent into a critical system vulnerability.

Main Research Question:

To what degree does increasing the density of exposed, low-privilege MCP tools accelerate the Attack Success Rate (ASR) of privilege aggregation exploits in edge-deployed, open-weight language models subjected to indirect prompt injection?

Sub-Research Questions:

1. **Baseline Competence:** Can sub-10B parameter models (e.g., Hermes-2-Pro, Qwen-2.5-Coder) maintain a >90% benign task completion rate when orchestrating complex JSON-RPC tool schemas, thus establishing a valid control state for adversarial testing?
2. **Vulnerability Scaling:** How do the Tool Invocation Deviation (TID) and Attack Success Rate (ASR) metrics fluctuate when the agent has access to 1 tool, versus 3 tools, versus 5 tools during an adversarial injection attempt?
3. **Architectural Fragility:** Do quantized open-weight models exhibit a uniform failure pattern across different types of tool schemas (e.g., read vs. write tools), or do specific combinations disproportionately trigger privilege aggregation?

Hypothesis / Expected Finding:

It is hypothesized that exposing three or more distinct tool classes via MCP significantly collapses the model's ability to differentiate between benign data tokens and malicious command tokens. This architectural conflation will increase the privilege aggregation exploit success rate by a statistically significant margin (an estimated >40% increase) compared to isolated, single-tool exposure scenarios.

Scope Boundaries (What the paper will NOT claim): The study explicitly bounds its scope to open-weight models under 10 billion parameters executed locally. It will *not* claim to evaluate the security posture of state-of-the-art proprietary frontier models (e.g., OpenAI's GPT-4o, Anthropic's Claude 3.5 Sonnet).[1] Additionally, the study will *not* address or audit server-side infrastructure code flaws (e.g., raw SQL injection or bash command vulnerabilities in Python server code, as comprehensively evaluated by VIPER-MCP [16]). The study rigorously isolates the *cognitive semantic susceptibility* of the LLM to tool-chain manipulation.

Proposed Methodology

To accommodate the zero-budget constraint and the absolute necessity of preventing empirical false negatives (caused by inherent model incompetence rather than adversarial resilience), the methodology follows a rigorous, phased experimental design utilizing a local, sandboxed environment.

Experimental Setup:

1. **Environment Isolation:** A Docker-containerized evaluation framework isolating the host machine's actual filesystem from the agent's executable workspace, ensuring that arbitrary code execution exploits do not damage the researcher's hardware.[1]
2. **MCP Architecture:** A custom Python-based MCP Client acting as the orchestration

loop. This client will connect to mock MCP servers built utilizing the official FastMCP library.[1] The servers will expose controlled, mock capabilities (e.g., reading a sandbox directory, querying a local SQLite database).

3. **The Threat Model Simulation:** The experiment will simulate an "External Attacker (Indirect)" scenario.[1] Adversarial markdown instructions—extracted directly from the AgentDojo benchmark—will be embedded in benign text files retrieved by the agent's reading tools.[4]

Variables:

- **Independent Variable:** Tool Density. This is the operationalization of privilege aggregation. The number of active mock MCP tools exposed to the agent will be scaled across three levels: Level 1 (1 isolated tool); Level 2 (3 concurrent tools); Level 3 (5 concurrent tools).[1]
- **Dependent Variables:**
  - *Attack Success Rate (ASR):* The percentage of experimental runs where the adversary successfully coerces the agent into executing a prohibited action (e.g., extracting data with Tool A and exfiltrating it via Tool B).
  - *Tool Invocation Deviation (TID):* A secondary metric measuring the ratio of unexpected tool calls to total expected tool calls, indicating the severity of context hijacking.[1]

Phase 1: The Baseline Competence Test (Minimum Viable Requirement) Before any adversarial payloads are introduced, the chosen open-weight model must be evaluated on 50 strictly non-adversarial tool-calling tasks (e.g., "Read local file X, summarize its contents, and save to file Y"). If the model fails to format the JSON-RPC correctly, hallucinate tool names, or fails to complete the tasks in >10% of cases, the model is rejected as an invalid test subject.[1] A baseline competence of >90% must be mathematically proven to ensure that subsequent failures during Phase 2 are legitimately attributable to the adversarial injection, not cognitive incapacity.

Phase 2: Privilege Aggregation Evaluation (The Core Experiment) Deploy indirect prompt injections using standardized test cases from the AgentDojo suite.[4] Execute the tasks while exposing the agent to Level 1 tool density. Record the ASR. Repeat the exact same prompts while exposing the agent to Level 2 and Level 3 tool densities. Record the shifting ASR and TID at each level.

Statistical Analysis & Reproducibility Plan: To ensure absolute reproducibility, all LLM inference temperatures will be strictly set to 0.0 (greedy decoding) to force deterministic, highly repeatable outputs.[29] The resulting ASR variance across the three tool density levels will be evaluated using an Analysis of Variance (ANOVA) test to determine statistical significance. The Dockerfiles, Python orchestration scripts, static mock datasets, and complete LLM interaction logs will be hosted on a public, anonymized GitHub repository linked within the final manuscript, allowing peer reviewers to execute the exact methodology with a single shell command.

Expected Charts / Tables:

1. A bar chart establishing the >90% baseline competence of the selected model.
2. A line graph demonstrating the primary finding: Attack Success Rate (Y-axis) scaling

upward as Tool Density (X-axis) increases from 1 to 5 tools.

3. A comprehensive Markdown table detailing the exact prompt injection vectors used (sourced from AgentDojo) and their corresponding success states.

Failure Fallback Plan:
If the primary model (e.g., Hermes-2-Pro) consistently fails the Phase 1 baseline test despite system prompt tuning, the fallback plan is to immediately substitute the model for a highly specialized coding model, such as Qwen-2.5-Coder-7B, which possesses significantly higher adherence to JSON structuring logic.

Dataset / Tools / Compute Plan

The entire suite of required materials fits perfectly within the financial constraints of the undergraduate student ($0.00). The methodology relies entirely on open-source ecosystems.

- **Compute Requirements:** A standard modern laptop (minimum 16GB Unified Memory / RAM) is sufficient. Inference will run exclusively on the CPU or local dedicated GPU utilizing the highly optimized Ollama inference engine.[1] Absolutely no cloud API keys (such as OpenAI or Anthropic developer tiers) are required.
- **Models to Test:** Highly quantized (GGUF format), tool-optimized small models specifically fine-tuned for function calling. General-purpose chat models (like standard Llama-3-8B-Instruct) must be strictly avoided to prevent JSON formatting failures.[1]
  - *Primary Target:* Hermes-2-Pro-Llama-3-8B
  - *Secondary Target:* Qwen-2.5-Coder-7B
- **Datasets & Benchmarks:**
  - *Primary Integration:* **AgentDojo**.[4] This peer-reviewed framework contains 629 security test cases specifically designed for agents executing tools over untrusted data. The researcher will extract the JSON-formatted prompt injection payloads from AgentDojo to feed into the local Python MCP framework, guaranteeing academic rigor and bypassing the need to synthesize custom datasets.
- **Libraries and Tooling:** Python 3.11, the official Anthropic MCP Python SDK (mcp and fastmcp), and Docker Desktop (Free Tier).[1]

Publication Venue Fit

To satisfy the mandate of no physical conference presentations, no mandatory travel, and a strict $0 Article Processing Charge (APC) via traditional subscription routes, the publication strategy avoids the traditional computer science conference circuit (e.g., USENIX, ACM CCS). Instead, it targets established, indexed hybrid journals that offer high prestige without financial barriers.

Venue Name	Publisher	Scope Fit & Indexing	APC Status & Presentation	Risk Level / Strategy
Journal of Information	Elsevier	**Excellent.** Focuses on	**$0 via Hybrid Route.** Authors	**Moderate-High.** Known for

Security and Applications (JISA) [1]		practice-driven applications of security. Bridges academia and industry. Indexed in Scopus and SCIE (Q1/Q2).	can select the traditional subscription route for free publication.[1] No presentation required.	relatively fast initial decision times (~3 weeks), which is highly ideal for undergraduate timelines.[1] This is the **Primary Target**.
Security and Privacy [1]	Wiley	**Strong.** Broad focus on network security and emerging technologies. Indexed in Scopus and ESCI (Q2/Q3).	**$0 via Hybrid Route.** Subscription route available with zero APC.[1] No presentation required.	**Moderate.** Serves as a highly reliable fallback with a slightly lower barrier to entry and reasonable review timelines (~21 days to first decision).[1]
Journal of Cyber Security Technology [1]	Taylor & Francis	**Good.** Encourages embedded system and threat detection research. Indexed in Scopus and ESCI (Q2).	**$0 via Hybrid Route.** Subscription route available.[1] No presentation required.	**Moderate.** Features an extremely fast initial decision (averaging 3 days) but post-review editorial processes can occasionally drag.[1]

Execution Strategy: Submit the manuscript to *JISA* (Elsevier) as the primary target. The paper must be formatted strictly in LaTeX utilizing the standard Elsevier double-column article class to ensure professional presentation. If JISA requests heavy revisions that exceed the undergraduate timeline, or issues an outright rejection, the researcher must immediately cascade the submission to Wiley's *Security and Privacy* to maintain momentum.

Risks and Mitigations

The execution of advanced AI security research by an undergraduate entails specific structural

risks. Implementing proactive mitigations is essential to prevent project failure.

Primary Risk	Severity	Mitigation Strategy
Local LLM Cognitive Failure	**CRITICAL**	If the 8B model cannot natively parse JSON-RPC, the ASR metrics are meaningless.[1] **Mitigation:** The Phase 1 Baseline Competence test is mandatory. Require a >90% benign task completion rate using tool-optimized models (Hermes-2-Pro).[1] Do not proceed to the attack phase until this mathematical baseline is proven.
Reviewer Demand for Proprietary Models	High	Reviewers at major journals frequently critique papers for not testing frontier models like GPT-4o. **Mitigation:** Aggressively frame the manuscript around "Edge-Deployed, Privacy-Preserving Agentic Systems" in the abstract and introduction. Argue structurally that local execution is a strict enterprise requirement for sensitive data [1], thereby justifying the use of small models as a distinct, vital research niche rather than a budget constraint.
Framework Obsolescence	Medium	The MCP standard is evolving rapidly; an update during the research phase

		could break the local code.[1] **Mitigation:** Freeze all dependency versions in a strict requirements.txt file and a finalized Docker image on Day 1. Frame the vulnerability within the paper as a fundamental *architectural flaw* (leveraging the Lethal Trifecta concept [3]), ensuring the research remains relevant regardless of minor software patches.
"Project Report" Tone	High	Undergraduate writing often lacks academic rigor, leading to immediate editorial desk rejections.[1] **Mitigation:** Strict adherence to academic prose. Use LaTeX. Emphasize statistical variance (ANOVA) and defined mathematical equations for ASR and TID to signal deep empirical rigor.

First 14-Day Execution Plan

The success of a time-constrained project relies on aggressive, front-loaded technical validation. The first two weeks must definitively prove the hardware and software stack can execute the experimental methodology.

- **Day 1-3 (Environment Freezing and Architecture Setup):**
  - Install Docker Desktop and Ollama on the local machine.
  - Download Hermes-2-Pro-Llama-3-8B (or equivalent tool-use model) via the Ollama CLI.
  - Create a Python virtual environment; execute pip install mcp and freeze the requirements immediately.
- **Day 4-7 (Mock Server Creation & Interfacing):**
  - Write a foundational FastMCP Python server exposing three highly basic, benign tools: read_file, get_weather, and write_log.
  - Write the Python client orchestrator that passes a user prompt to the local LLM and

feeds the LLM's JSON-RPC output back to the MCP server. Ensure standard output logging is operational.

- **Day 8-11 (The Competence Baseline - KILL CRITERIA):**
  - Execute 50 basic, benign tasks using the orchestrator (e.g., "Read the file in directory X and log the output to directory Y").
  - **Kill Criteria:** Evaluate the logs. If the model fails to achieve a >90% completion rate due to persistent JSON formatting errors or severe tool hallucinations, stop immediately. The project cannot proceed until the system prompt is tuned or a new local model (e.g., Qwen-Coder) is found that successfully passes the baseline. Proceeding with an incompetent model guarantees unpublishable data.
- **Day 12-14 (Benchmark Extraction and First Injection):**
  - Clone the open-source AgentDojo repository.[4]
  - Extract 20 JSON-based indirect prompt injection payloads specifically related to multi-tool manipulation.
  - Inject the first adversarial payload into a mock read_file document and observe the LLM's state transition to verify the attack surface is active.

Final Go / No-Go / Revise Verdict

Verdict: GO (Conditional upon strict execution of the revised scope).
Main Reason: By abandoning the overly broad, dual-focus approach on both transitive trust and privilege aggregation, and pivoting exclusively to a focused quantitative measurement of privilege aggregation in edge-deployed models utilizing established benchmarks, this project transforms from a risky conceptual proposal into a highly feasible, scientifically rigorous experiment that requires exactly $0.00 in funding.
Top 3 Strengths:

1. **Unsaturated Academic Niche:** While industry reports on MCP are rising rapidly [7], formal mathematical evaluations of cognitive privilege aggregation under tool-scaling pressure remain virtually absent in indexed academic journals.
2. **Zero-Cost Execution:** The entire testing apparatus, from LLM inference to containerized sandboxing, can run deterministically on a standard consumer laptop via Docker and Ollama, perfectly aligning with the constraints.[1]
3. **Clear Publication Pathway:** Utilizing hybrid publishing models at top-tier publishers like Elsevier bypasses all financial (APC) and travel barriers while maintaining high academic prestige and SCIE/Scopus indexing.[1]

Top 3 Risks:

1. Small-parameter model incompetence rendering the empirical data statistically useless (mitigated by the Phase 1 Baseline).
2. Reviewer prejudice against empirical evaluations that do not utilize state-of-the-art proprietary models like GPT-4 (mitigated by explicit narrative framing around edge-deployed architectures).
3. Failure to strictly isolate the execution environment, potentially leading to compromised host systems during the testing of malicious payloads.

Required Revisions Implemented: The scope has been permanently narrowed from a general

"Agentic Security" overview to the exact empirical measurement of "Privilege Aggregation Vulnerabilities." The highly risky endeavor of synthesizing custom datasets has been completely abandoned in favor of extracting validated payloads from the AgentDojo framework.
If the 14-day baseline competence test yields a >90% success rate on benign tasks, the researcher is unequivocally cleared to execute the full adversarial methodology.

Works cited

1. AI Research Viability and Publication Strategy MCP.pdf
2. Trustworthy Agentic AI Requires Deterministic Architectural Boundaries - arXiv, accessed June 23, 2026, https://arxiv.org/html/2602.09947v1
3. [2602.09947] Trustworthy Agentic AI Requires Deterministic Architectural Boundaries - arXiv, accessed June 23, 2026, https://arxiv.org/abs/2602.09947
4. [2406.13352] AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents - arXiv, accessed June 23, 2026, https://arxiv.org/abs/2406.13352
5. From AI-Generated Content to Agentic Action: Security and Safety Threats in Generative AI - arXiv, accessed June 23, 2026, https://arxiv.org/pdf/2605.16471
6. Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents - arXiv, accessed June 23, 2026, https://arxiv.org/html/2601.17549v1
7. Agentjacking: Sentry MCP Injection Hijacks AI Coding Agents - Lab Space, accessed June 23, 2026, https://labs.cloudsecurityalliance.org/research/csa-research-note-agentjacking-sentry-mcp-20260614-csa-style/
8. WILEY READ AND PUBLISH AGREEMENT DEFINITIONS, accessed June 23, 2026, https://consortium.ch/wp_live/wp-content/uploads/2021/05/Wiley_Journals_agreement_2021-2024.pdf
9. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents - arXiv, accessed June 23, 2026, https://arxiv.org/pdf/2410.02644
10. [2601.17549] Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents - arXiv, accessed June 23, 2026, https://arxiv.org/abs/2601.17549
11. Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents - arXiv, accessed June 23, 2026, https://arxiv.org/pdf/2601.17549
12. [2603.22489] Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning - arXiv, accessed June 23, 2026, https://arxiv.org/abs/2603.22489
13. Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning - arXiv, accessed June 23, 2026, https://arxiv.org/html/2603.22489v1
14. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents, accessed June 23, 2026, https://arxiv.org/html/2406.13352v1

15. Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks? - arXiv, accessed June 23, 2026, https://arxiv.org/html/2510.05244v1
16. VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers - arXiv, accessed June 23, 2026, https://arxiv.org/html/2605.21392v1
17. [2605.21392] VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers - arXiv, accessed June 23, 2026, https://arxiv.org/abs/2605.21392
18. VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers - arXiv, accessed June 23, 2026, https://arxiv.org/pdf/2605.21392
19. Prompt Injection Attack to Tool Selection in LLM Agents | Request PDF - ResearchGate, accessed June 23, 2026, https://www.researchgate.net/publication/404837831_Prompt_Injection_Attack_to_Tool_Selection_in_LLM_Agents
20. Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis of Vulnerabilities in Skills, Tools, and Protocol Ecosystems - arXiv, accessed June 23, 2026, https://arxiv.org/pdf/2601.17548
21. Tenet's 'Agentjacking' Attack Turns Sentry Errors Into Code Execution, accessed June 23, 2026, https://devops.com/tenets-agentjacking-attack-turns-sentry-errors-into-code-execution/
22. When Agents Handle Secrets: A Survey of Confidential Computing for Agentic AI - arXiv, accessed June 23, 2026, https://arxiv.org/html/2605.03213v1
23. How Brittle is Agent Safety? Rethinking Agent Risk under Intent Concealment and Task Complexity - arXiv, accessed June 23, 2026, https://arxiv.org/html/2511.08487v1
24. ComplexMCP: Evaluation of LLM Agents in Dynamic, Interdependent, and Large-Scale Tool Sandbox - arXiv, accessed June 23, 2026, https://arxiv.org/html/2605.10787v1
25. AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System - arXiv, accessed June 23, 2026, https://arxiv.org/html/2602.03117v1
26. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents - arXiv, accessed June 23, 2026, https://arxiv.org/html/2403.02691v3
27. ComplexMCP: Evaluation of LLM Agents in Dynamic ... - Takara TLDR, accessed June 23, 2026, https://tldr.takara.ai/p/2605.10787
28. Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies - arXiv, accessed June 23, 2026, https://arxiv.org/pdf/2504.08623
29. SecInfer: Preventing Prompt Injection via Inference-time Scaling - arXiv, accessed June 23, 2026, https://arxiv.org/pdf/2509.24967
30. Dr. Muhammad Faraz Hyder ACADEMIC QUALIFICATION PROFESSIONAL EXPERIENCE RESEARCH PUBLICATIONS - NEDUET, accessed June 23, 2026, https://se.neduet.edu.pk/sites/default/files/SE/CV/Dr.%20M.Faraz_CV__New%20fo

rmat_April_2025.pdf
31. AI-DRIVEN FRAMEWORK FOR SCALABLE, SECURE, AND INTELLIGENT BIG DATA MANAGEMENT IN CLOUD ENVIRONMENTS, accessed June 23, 2026, https://kjmr.com.pk/kjmr/article/view/949
32. Engineering & All HJRS Categories | PDF - Scribd, accessed June 23, 2026, https://www.scribd.com/document/711842174/Engineering-All-HJRS-Categories
33. T&F_Journals_TA_2024-2027 copy - California Digital Library, accessed June 23, 2026, https://cdlib.org/services-groups/collections/licensed_resources/redacted_licenses/T%26F_Journals_TA_2024-2027_Redacted.pdf